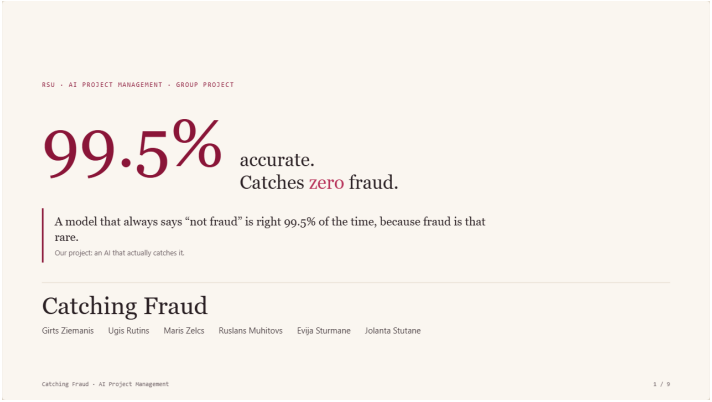
SLIDE 1 / 9 · RSU · AI PROJECT MANAGEMENT · GROUP PROJECT

Hook



WHAT TO SAY

- (~20 s) "Imagine a fraud detector that is 99.5% accurate. Sounds great? It catches zero fraud."
- Only 0.52% of card payments are fraud, so a model that always says "not fraud" is right 99.5% of the time and useless.
- "We are team Catching Fraud, and in the next minutes we show you an AI that does catch it: where its data comes from, why it needs AI, and how we'd launch it."

BACKGROUND

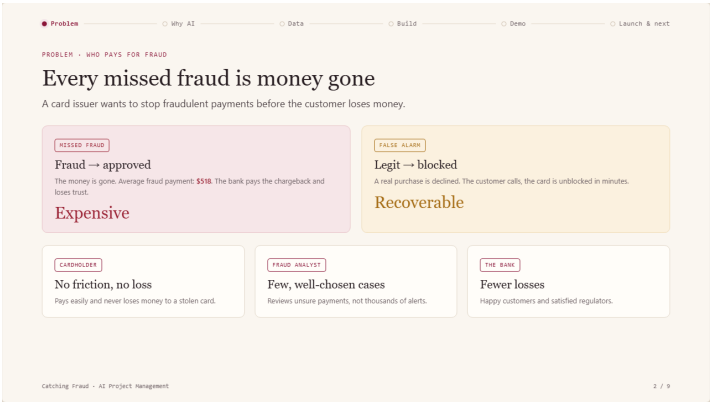
Why accuracy misleads here. Only 1,782 of 339,607 payments in our data are fraud (0.52%). A model that answers “not fraud” every time is therefore correct 99.48% of the time, yet it never stops a single fraudster. Accuracy counts every correct answer equally, so when one class is tiny the big class dominates the score.

What we measure instead. From the start we defined success as *Recall* (the share of real fraud we catch), with false alarms as a guardrail (we must not block every customer) and money as the business measure. Every number later in the pitch is one of these three.

The storyline. Problem → why AI → where the data comes from → what fraud looks like → how we built and chose the model → pipeline and demo → how we would launch it. The tracker at the top of each slide shows where we are.

SLIDE 2 / 9 · PROBLEM · WHO PAYS FOR FRAUD

The problem



WHAT TO SAY

- (~40 s) Two possible mistakes, very different prices. A missed fraud is real money lost (average $518 in our data) plus a chargeback and lost trust. A false alarm is an annoyed customer and a phone call.
- So our goal is: catch as much fraud as possible, while keeping false alarms low. "When in doubt, stop the transaction."
- Three users: the cardholder, the fraud analyst who reviews cases, and the bank that pays for losses.

BACKGROUND

Two kinds of mistakes. A *missed fraud* (false negative) means a fraudulent payment is approved. A *false alarm* (false positive) means a genuine payment is blocked. A fraud model always trades one against the other: catching more fraud usually means more false alarms.

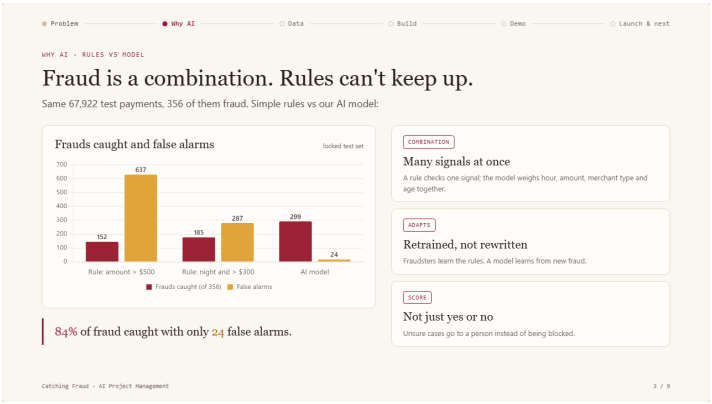
Why the prices differ. The average fraudulent payment in our data is $518 (median $356), compared with $68 for a normal payment. On top of the lost money the bank handles a chargeback and risks losing the customer's trust. A false alarm costs a phone call and some annoyance, which is recoverable, but too many of them drive customers away. That is why false alarms are a guardrail, not something we ignore.

Three users. The cardholder wants to pay without friction and never lose money. The fraud analyst needs a small number of well-chosen cases to review, not thousands of alerts. The bank pays for every missed fraud and must keep customers and regulators satisfied.

Course link. Lecture 6 makes the same point with a different example: “a wrong blur is cheap; a missed face is not.” This cost difference drives every later decision: the metric, the model choice, the threshold and the score bands.

SLIDE 3 / 9 · WHY AI · RULES VS MODEL

Why AI



WHAT TO SAY

- (~45 s) Why do we need AI at all? We tested the obvious rules on exactly the same test payments as the model.
- "Block every payment over $500" catches 152 of 356 frauds (43%) and blocks 637 innocent customers. "Block night payments over $300" catches 185 (52%) with 287 false alarms.
- Our AI model catches 299 (84%) with only 24 false alarms. It combines many weak signals that a rule looks at one by one.
- And fraud changes: rules must be rewritten by hand, a model is retrained. It also gives a score, so unsure cases can go to a person.

BACKGROUND

How we tested the rules. We applied two simple rules to exactly the same locked test set as the model: 67,922 payments, 356 of them fraud. “Block every payment over $500” catches 152 frauds (43%) but blocks 637 genuine customers. “Block night payments (22:00–03:59) over $300” catches 185 (52%) with 287 false alarms. Our AI model catches 299 (84%) with only 24 false alarms.

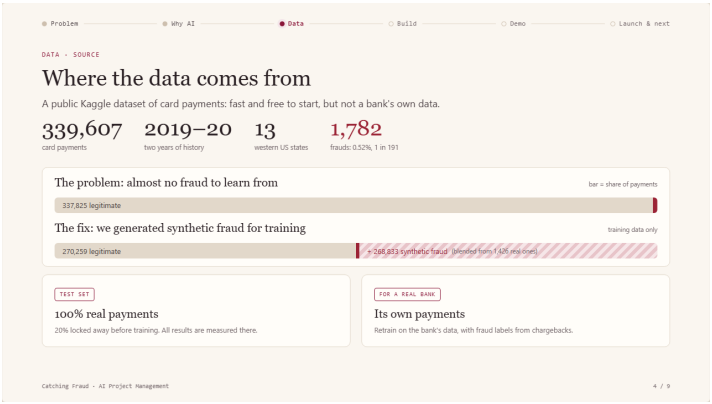
Why the model wins. A rule looks at one or two signals and draws a hard line. Fraud rarely shows up in one signal; it shows up in combinations, for example a mid-sized online purchase late at night by an older customer. The model weighs all 39 inputs together and learns these combinations from past payments.

Why it matters over time. Fraudsters adapt to whatever gets blocked. A rule set has to be rewritten by hand; a model is retrained on newly confirmed fraud. The model also outputs a score from 0 to 1 instead of yes/no, which lets unsure cases go to a person.

Honest caveat. These are simple illustrative rules, not a bank's tuned rule engine. Real rule sets are stronger, but they hit the same limit: every rule is written by a person and checks a fixed combination.

SLIDE 4 / 9 · DATA · SOURCE

Where the data comes from



WHAT TO SAY

- (~50 s) Our data: a public Kaggle dataset, 339,607 card payments from 13 western US states, 2019–2020. Lecture 2: an existing dataset is cheap and ready today, but it was collected by someone else for someone else.
- The core challenge: only 1,782 frauds, 1 in 191. A model hardly sees any fraud to learn from.
- Our fix: generated data. SMOTE creates new fraud examples by blending similar real frauds, until the training data is 50/50: 268,833 synthetic frauds made from 1,426 real ones. We also tested the alternative, class weights, where rare fraud simply counts more.
- The test set (67,922 payments) stays 100% real and untouched, so every result we show is on real payments.
- For a real bank: retrain on its own payments; the fraud label then comes from chargebacks weeks later. Check whether the Kaggle data is simulated, and say so if asked.

BACKGROUND

The dataset. Public Kaggle dataset "credit-card-fraud-dataset" (dhruvb2028): 339,607 card payments from 2019–2020 in 13 western US states, 14 merchant categories, 15 columns per payment (time, merchant, category, amount, city, state, customer location, city population, job, date of birth, transaction ID, merchant location, fraud label), with no missing values.

The source decision (Lecture 2). Of the four data sources in Lecture 2 (existing datasets, own collection, video, generated), we used an existing dataset: low cost and ready today, but collected by someone else for someone else. A real bank would retrain on its own payments, with fraud labels that arrive weeks later through chargebacks. If asked whether the data is simulated, check the Kaggle page before answering.

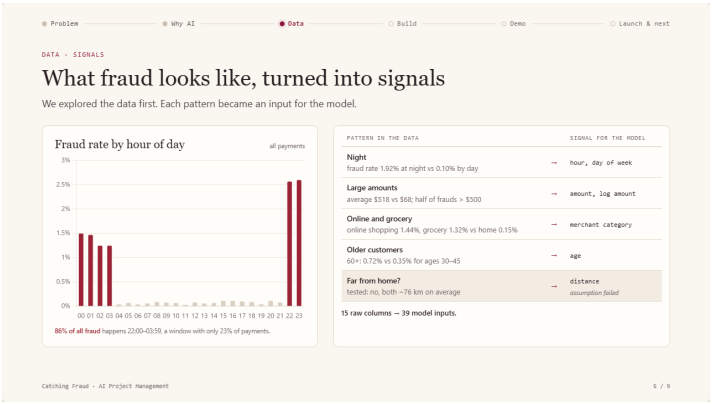
The split. Before any modelling, 20% was locked away as the test set, stratified to keep the same fraud ratio and with a fixed random seed (42) so it can be rebuilt. Training set: 271,685 payments with 1,426 frauds. Test set: 67,922 payments with 356 frauds. All results in the pitch come from this real, untouched test set.

How the synthetic fraud is made (SMOTE). For a real fraud case, SMOTE picks one of its five most similar fraud cases and creates a new example somewhere on the line between them. Repeating this until both classes are equal gave 270,259 legitimate and 270,259 fraud examples, so 268,833 fraud examples are synthetic. Only the training data was balanced.

Two approaches tested. SMOTE was used for Logistic Regression, Decision Tree and Random Forest. XGBoost and LightGBM used class weights instead: no data is generated, the rare fraud cases simply count more during training. Lesson learned: because SMOTE ran before cross-validation, synthetic copies leaked into the validation folds and made those scores look too good (Random Forest 0.999 in validation vs 0.768 on the test set). The fix is to generate synthetic data inside each fold. Lecture 2 warns about this: generated data is cheap but may not look real.

SLIDE 5 / 9 · DATA · SIGNALS

What fraud looks like



WHAT TO SAY

- (~45 s) Before building any model, we looked at what makes fraud different, and turned each finding into a signal.
- Night: 86% of fraud happens between 22:00 and 04:00, a window with only 23% of payments. Amount: fraud averages $518 vs $68. Category: online shopping and grocery have the highest fraud rates. Age: customers over 60 are targeted twice as often as 30–45 year olds.
- We also tested an assumption: "fraud happens far from home". The data said no: both are about 76 km. Lesson: test assumptions with data.

BACKGROUND

From pattern to signal. Before training anything we explored the data to see what makes fraud different, then turned each finding into an input the model can use. This step is called feature engineering: 15 raw columns became 39 model inputs. All numbers on this slide are computed from the full dataset.

Night. 86% of all fraud happens between 22:00 and 03:59, a window with only 23% of payments. The fraud rate at night is 1.92% against 0.10% in the daytime, about 19 times higher. Signals: hour and day of week, extracted from the timestamp.

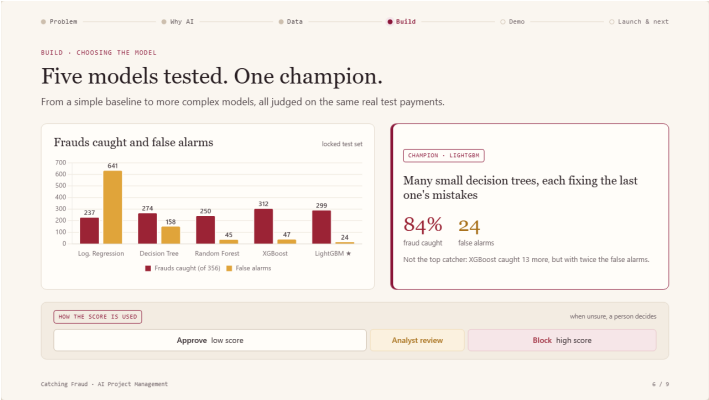
Large amounts. Fraud averages $518 against $68 for normal payments (medians $356 vs $46); 47% of frauds are above $500. Signals: the amount and its logarithm, which compresses the range from $1 to over $28,000 so very large values do not dominate.

Merchant category and age. Online shopping (1.44%), in-store grocery (1.32%) and online miscellaneous (1.28%) have the highest fraud rates, against 0.15% for home goods; each category becomes a yes/no column. Customers over 60 have a fraud rate of 0.72%, double the 0.35% of 30–45 year olds; age is calculated from date of birth at the time of the payment.

The assumption that failed. We expected fraud to happen far from the cardholder's home. We calculated the distance between home and merchant for every payment: the average is 75.6 km for both fraud and normal payments. We kept the feature, but the lesson is clear: test assumptions with data instead of trusting intuition.

SLIDE 6 / 9 · BUILD · CHOOSING THE MODEL

How we built it



WHAT TO SAY

- (~50 s) Lecture 3: start simple, change one idea at a time. We trained five models, from a simple straight-line model to tree ensembles, tuned each with cross-validation and judged all on the same locked test set.
- Baseline (Logistic Regression) caught 237 frauds with 641 false alarms. The champion, LightGBM, caught 299 (84%) with only 24 false alarms.
- We chose on balance, not on the single highest number: XGBoost caught 13 more but with 47 false alarms. Honest note: with only 356 frauds, a gap of 13 is within the noise (about ±14), so it's a business choice to confirm on live data.
- The model outputs a score. Low: approve. High (0.4 and up): block. In between (0.1–0.4): a fraud analyst decides. On our test set that middle band holds only about 23 payments, 6 of them fraud.
- Details in the backup deck: experiment log, threshold tuning, and the lesson that synthetic data made validation look too good.

BACKGROUND

The approach (Lecture 3). Start with a simple baseline and add complexity one idea at a time, so each improvement can be explained. Logistic Regression tests a single straight-line rule; Decision Tree tests yes/no rule combinations; Random Forest tests a vote of many trees; XGBoost tests trees that correct each other's mistakes; LightGBM tests the same idea, built for speed. Each model was tuned automatically with 5-fold cross-validation on the training data only, then judged once on the locked test set.

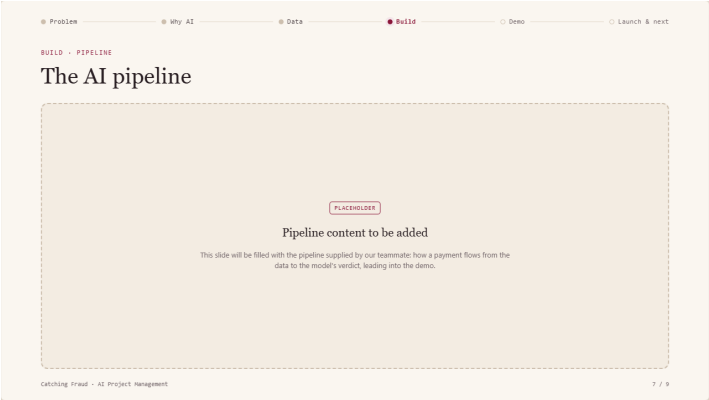
Results on the test set (Recall / precision / F1): Logistic Regression 0.67 / 0.27 / 0.38; Decision Tree 0.77 / 0.63 / 0.70; Random Forest 0.70 / 0.85 / 0.77; XGBoost 0.88 / 0.87 / 0.87; LightGBM 0.84 / 0.93 / 0.88, with ROC-AUC 0.998. Precision is the share of alarms that were real fraud; F1 balances Recall and precision.

Why LightGBM. It has the best balance (highest F1) and the fewest false alarms: 24, against 47 for XGBoost. XGBoost caught 13 more frauds, but with only 356 fraud cases our Recall is uncertain by about ±4 points (≈ ±14 frauds), so a gap of 13 could be luck. We treated the choice as a business trade-off to confirm on live data, not as a proven win.

From score to decision. The model outputs a fraud score. The default cutoff is 0.5; lowering it to 0.4 catches 304 instead of 299 frauds for 28 instead of 24 false alarms and gives the highest F1 (0.884). We propose three bands: approve below 0.1, block at 0.4 and above, and send the band in between to a fraud analyst. On the test set that middle band holds only about 23 payments, 6 of them fraud, a tiny workload with high value (Lecture 5: a rule for “not sure”; Lecture 6: when unsure, a person decides).

SLIDE 7 / 9 · BUILD · PIPELINE

AI pipeline



WHAT TO SAY

- (~45 s) Placeholder: pipeline content and speaker notes to be provided by teammate.
- End by handing over to the demo: "Let's see it in action."

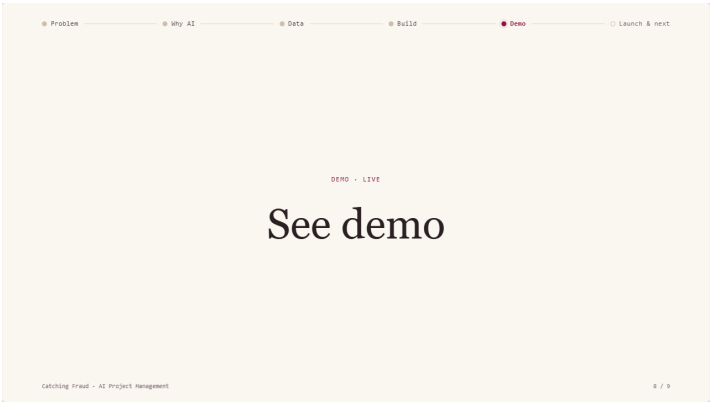
BACKGROUND

Placeholder. Background material for this slide will be provided by the teammate presenting the pipeline.

Suggested structure (Lecture 6). Train → export → catalogue → build → deploy → serve → watch, with each step marked as done in our project or planned for production.

SLIDE 8 / 9 · DEMO · LIVE

Live demo



WHAT TO SAY

- (~1:30) Switch to the demo window (Flask API on port 5050, Next.js dashboard on port 3000).
- Suggested order: an everyday purchase (approved), a payment with the fraud fingerprint: night, large amount, online (blocked), a borderline case (show how the score decides).
- Backup plan: have screenshots ready in case the demo fails.

BACKGROUND

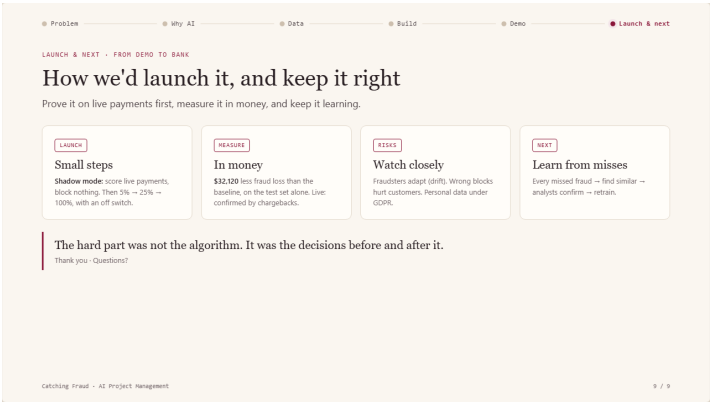
What runs behind the demo. The trained LightGBM model is saved together with the scaler and the exact list and order of the 39 input columns. A Flask API (port 5050) loads these files and scores one payment at a time; a Next.js dashboard (port 3000) generates test payments, sends them to the API and shows the verdict. Both run together with Docker Compose.

Why the saved column list matters. If the API built its inputs in a different order than during training, every prediction would be silently wrong with no error message. Lecture 2 lists this kind of silent pitfall (for example a renumbered class list). Saving the feature list with the model prevents it.

Preparation. Test the three demo cases on the presenting laptop beforehand and note the exact inputs that give approve, block and borderline results. Keep screenshots ready as a fallback, and keep the demo to about 90 seconds.

SLIDE 9 / 9 · LAUNCH & NEXT · FROM DEMO TO BANK

Launch and next steps



WHAT TO SAY

- (~50 s) Launch (Lecture 8): first shadow mode, where the model scores real payments but blocks nothing. Fraud is confirmed by chargebacks up to ~60 days later, so we wait before judging. Then a canary release: 5%, 25%, 100% of payments, always with a way back.
- Measure: success in money. On the test set, the champion prevents about $32,120 more fraud loss than the baseline model (62 more frauds × $518). Live, we'd also track false alarms, complaints and results per customer group.
- Risks: fraudsters change tactics, so the model ages; real customers wrongly blocked; personal data (birth date, location) under GDPR.
- Next (Lecture 2, data engine): each missed fraud becomes training data for the next model.
- Close with the one-liner, then open for questions. The backup deck has all details.

BACKGROUND

Launch in small steps (Lecture 8). A test score is only a promise: in the SickKids silent trial a model scored 0.90 on old data but 0.50 on new patients. So we would first run in shadow mode, scoring every real payment but blocking nothing. Fraud is only confirmed when chargebacks arrive, up to about 60 days later (Stripe has the same delay), so we wait before judging. Then a canary release: 5%, 25%, then 100% of payments decided by the model, compared with today's rules in the same weeks, with an off switch at every step.

Measure in money. On the test set the champion catches 62 more frauds than the baseline model; at the average fraud value of $518 that is about $32,120 less loss on 67,922 payments. Live, we would track confirmed fraud caught, false alarms and complaints, losses prevented per month, and results per customer group. Because a blocked payment never tells us whether it was really fraud, a person reviews a small random sample of blocks each week.

Main risks. Drift: fraudsters change tactics, so the model ages and must be retrained. Wrong blocks hurt real customers. Unequal errors: age is one of the model's strongest signals, so error rates must be checked per age group. Privacy: the data contains birth dates, jobs and home locations, which falls under GDPR (access by role, deletion on request).

Next steps (Lecture 2, the data engine). Every missed fraud starts a loop: find similar payments, let analysts confirm them, retrain, and compare with the current champion. First targets: study the 57 frauds the model missed, version the data as well as the model, and retrain on the bank's own payments.

Closing line. “The hard part was not the algorithm. It was the decisions before and after it.” Then open for questions; the backup deck has every detail.